LAB# 10
Database connectivity with MY SQL

Introduction

To access and add content to a **MySQL** database, you must first establish a connection between the database and a **PHP** script.
In this lecture, learn how to use MySQLi Extension and PHP Data Objects to connect to MySQL.

Prerequisites

- Special CREATE privileges
- A MySQL Database
- A MySQLi or PDO extension

2 Ways to Connect to MySQL database using PHP

There are two popular ways to connect to a MySQL database using PHP:

1. With **PHP's MySQLi Extension**
2. With **PHP Data Objects (PDO)**

The guide also includes explanations for the credentials used in the PHP scripts and potential errors you may come across using MySQLi and PDO.

Option 1: Connect to MySQL with MySQL Improved extension

MySQLi is an extension that only supports MySQL databases. It allows access to new functionalities found in MySQL systems (version 4.1. and above), providing both an object-oriented and procedural interface. It supports server-side prepared statements, but not client-side prepared statements.
The MySQLi extension is included PHP version 5 and newer.
The PHP script for connecting to a MySQL database using the MySQLi procedural approach is the following:

```
<?php
$servername = "localhost";
$database = "database";
$username = "username";
$password = "password";
// Create connection
$conn = mysqli_connect($servername, $username, $password, $database);
// Check connection
if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
```

}
echo “Connected successfully”;
mysqli_close($conn);
?>

Credentials Explained

The first part of the script is four variables (server name, database, username, and password) and their respective values. These values should correspond to your connection details.

```
$servername = "localhost";
$database = "database";
$username = "username";
$password = "password";
```

Next is the main PHP function **mysqli_connect()**. It establishes a connection with the specified database.

```
// Create connection

$conn = mysqli_connect($servername, $username, $password, $database);
```

Following is an “**if statement**.” It is the part of the code that shows whether the connection was established. When the connection fails, it gives the message ***Connection failed***. The **die** function prints the message and then **exits** out of the script.

```
if ($conn->connect_error) {

die("Connection failed: " . $conn->connect_error);
}
```

If the connection is successful, it displays “**Connected successfully**.”

```
echo “Connected successfully”;
```

When the script ends, the connection with the database also closes. If you want to end the code manually, use the **mysqli_close** function.

```
mysqli_close($conn);

?>
```

Option 2: Connect To MySQL With PDO

PHP Data Objects (PDO) is an extension that serves as an interface for connecting to databases. Unlike MySQLi, it can perform any database functions and is not limited to MySQL. It allows flexibility among databases and is more general than MySQL. PDO supports both server and client-side prepared statements.

The PHP code for connecting to a MySQL database through the PDO extension is:

```
<?php
$servername = "localhost";
$database = "database";
$username = "username";
$password = "password";
$charset = "utf8mb4";
try {
$dsn = "mysql:host=$servername;dbname=$database;charset=$charset";
$pdo = new PDO($dsn, $username, $password);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

echo “Connection Okay”;

return $pdo

}

catch (PDOException $e)
{
echo “Connection failed: ”. $e->getMessage();
}
?>
```

Credentials Syntax

First, we have five variables (server name, database, username, password, and charset) and their values. These values should correspond to your connection details.

The **server name** will be *localhost*. If connected to an online server, type in the server name of that server.
The variable **charset** tells the database in which encoding it will be receiving and sending data. The recommended standard is **utf8mb4**.

```
$servername = "localhost";
$database = "database";
$username = "username";
$password = "password";
$charset = "utf8mb4";
```

Try and Catch Blocks

PDO's great asset is that it has an **exception** class to take care of any potential problems in database queries. It solves these problems by incorporating ***try and catch*** blocks.
If a problem arises while trying to connect, it stops running and attempts to **catch** and solve the issue. *Catch* blocks can be set to show **error messages** or run an **alternative code**.

```
try {
    $dsn = "mysql:host=$servername;dbname=$database;charset=$charset";
    $pdo = new PDO($dsn, $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connection Okay";

    return $pdo

}
catch (PDOException $e)
```

The first parameter in the *try and catch* block is **DSN,** which stands for **data(base) source name**. It is crucial as it defines the type and name of the database, along with any other additional information.
In this example, we are using a MySQL database. However, PDO supports various types of databases. If you have a different database, replace that part of the syntax (*mysql*) with the database you are using.

```
try {
    $dsn = "mysql:host=$servername;dbname=$database;charset=$charset";
```

Next is the **PDO** variable. This variable is going to establish a connection to the database. It has three parameters:

1. The data source name (dsn)
2. The username for your database
3. The password for your database

```
10    $pdo = new PDO($dsn, $username, $password);
```

Following is the **setAttribute method** adding two parameters to the PDO:

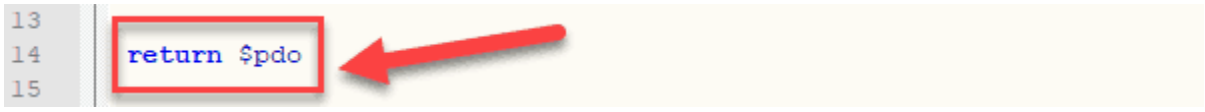
1. **PDO::ATTR_ERRMODE**
2. **PDO::ERRMODE_EXCEPTION**

This method instructs the PDO to run an **exception** in case a query fails.

Add the echo "**Connection Okay**." to confirm a connection is established.

```
12    echo "Connection Okay";
```

Return the PDO variable to connect to the database.



After returning the PDO variable, define the **PDOException** in the **catch** block by instructing it to display a message when the connection fails.

```
17  catch (PDOException $e)
18  {
19    echo "Connection failed: " . $e->getMessage();
```

Potential Errors with MySQLi and PDO

Incorrect Password

The password in the PHP code needs to correspond with the one in the database. If the two do not match, a connection with the database cannot be established. You will receive an error message saying *the connection has failed*.

Possible solutions:

1. Check the database details to ensure the password is correct.
2. Ensure there is a user assigned to the database.

Unable to Connect to MySQL Server

PHP may not be able to connect to the MySQL server if the server name is not recognized. Make sure that the **server name** is set to *localhost*.

In case of other errors, make sure to consult the ***error_log*** file to help when trying to solve any issues. The file is located in the same folder where the script is running.

Conclusion

This guide detailed two ways to connect to a MySQL database using PHP.
Both **MySQLi and PDO** have their advantages. However, bear in mind that MySQLi is only used for MySQL databases. Therefore, if you want to change to another database, you will have to rewrite the entire code. On the other hand, PDO works with 12 different databases, which makes the migration much easier.